

Week 7 Exercises

Exercise 1 `brm()`

Reproduce the familiar negative binomial regression below with `brm()`.

Hints:

1. See `?brms::brmsfamily` for the distributions available in `brm()`.
2. Use `#| results: hide` in your Quarto code chunks to hide the Stan junk output. It's pesky.

```
# load packages
library(glmmTMB)

# load data
holland <- crdata::holland2015

# formula corresponds to model 1 for each city in holland (2015) table 2
f <- operations ~ lower + vendors + budget + population

# fit poisson regression model for Santiago
fit <- glmmTMB(
  f,
  family = nbinom2,
  data = holland,
  subset = city == "santiago"
)

# summary
summary(fit)
```

Solution

```
# load packages
library(brms)

# fit poisson regression model for Santiago
fit_brm <- brm(
  f,
  family = negbinomial,
  data = subset(holland, city == "santiago") # brm() doesn't have subset
)
```

```
# summary
summary(fit_brm)
```

```
Family: negbinomial
Links: mu = log
Formula: operations ~ lower + vendors + budget + population
Data: subset(holland, city == "santiago") (Number of observations: 34)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000
```

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	2.79	2.23	-1.23	7.44	1.00	2135	1431
lower	-0.05	0.03	-0.13	0.01	1.00	2134	1290
vendors	-0.23	0.32	-0.87	0.41	1.00	1818	2045
budget	0.00	0.00	-0.00	0.01	1.00	2896	2587
population	0.04	0.05	-0.04	0.16	1.00	1732	1629

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.30	0.11	0.14	0.57	1.00	1949	2010

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

Exercise 2 Convergence

Using the output above, argue that `warmup` and `iter` are sufficiently large (or not).

Solution.

- **Setup recap**

4 chains; iter = 2000; warmup = 1000; post-warmup = 1000 per chain → 4000 total draws.

- **$\hat{R} \rightarrow$ all parameters near 1.00**

Why it matters: $\hat{R} \approx 1.00$ indicates chains mix to the same target after adaptation.

Evidence: Intercept, lower, vendors, budget, population, and shape all show `Rhat = 1.00`, consistent with well-tuned warmup and sufficient iterations for convergence.

- **Effective sample sizes $\rightarrow$ large**

Why it matters: Bulk_ESS gauges precision of means; Tail_ESS gauges interval tails. Rules of thumb: ESS 2000 per parameter. *Evidence:* Bulk_ESS and Tail_ESS are about 1900 to 2800 across parameters (e.g., lower: Bulk_ESS 2743; shape: 2240),

Exercise 3 `brm()` arguments

Explain the following arguments to `brm()`. Explain what the argument controls and recommend reasonable defaults.

- `chains`
 - `iter`
 - `warmup`
 - `cores`
 - `backend`
-

Solution.

- **chains**

What it controls: Number of independent MCMC chains to run.

Why it matters: Multiple chains start from different initial values, helping check convergence (e.g., via $\hat{R}$).

Reasonable default: `chains = 4`. Fewer if slow, more if diagnostics are uncertain.

- **iter**

What it controls: Total iterations per chain, including warmup.

Why it matters: More iterations yield more effective posterior draws and reduce Monte Carlo error.

Reasonable default: `iter = 2000` (with half used for warmup). Use 4000–8000 for complex models.

- **warmup**

What it controls: Number of iterations per chain used for tuning (e.g., step size, mass matrix) and discarded afterward.

Why it matters: Proper adaptation improves sampling efficiency and stability.

Reasonable default: `warmup = 1000` (half of `iter`).

- **cores**

What it controls: Number of CPU cores used to run chains in parallel.

Why it matters: Parallel chains shorten total runtime.

Reasonable default: `cores = chains`, often with `options(mc.cores = parallel::detectCores())`.

- **backend**

What it controls: The Stan interface used for sampling.

Options: `"cmdstanr"` or `"rstan"`.

Why it matters: Determines compilation speed, diagnostics, and toolchain.

Reasonable default: `"cmdstanr"` (if installed); otherwise defaults to `"rstan"`.

Example

```
fit <- brm(  
  y ~ x1 + x2,  
  data = df,  
  family = gaussian(),  
  chains = 4,  
  iter = 4000,  
  warmup = 2000,  
  cores = 4,  
  backend = "cmdstanr"  
)
```

Exercise 4 {rstan} or {cmdstanr}

Reproduce the fits from Exercise 1 using a `.stan` model via {rstan} or {cmdstanr}.

Hint: You might want to use `neg_binomial_2_log`—see the Stan docs.

Solution

First, we can create the data objects and list that {rstan} needs. You could also use {cmdstanr}—the logic is similar.

```
# load packages
library(rstan)

# create data object for stan
holland <- crdata::holland2015
sant <- subset(holland, city == "santiago")

# formula to create design matrix
operations ~ lower + vendors + budget + population

# data for stan
mf <- model.frame(f, data = sant)
X <- model.matrix(f, data = mf)
y <- model.response(mf)
N <- nrow(X)
K <- ncol(X)

# combine data for stan into single list
stan_data <- list(N = N, K = K, X = X, y = y)
```

Now we create the `.stan` file. You can write this in a separate file, but I'm including it as a character string in this document.

```
stan_code <- "
data {
  int<lower=1> N;
  int<lower=0> y[N];
  int<lower=1> K;
  matrix[N, K] X;
```

```

}
parameters {
  vector[K] beta;          // regression coefficients (includes intercept)
  real<lower=0> theta;      // shape/overdispersion
}
model {
  // weakly informative priors
  beta ~ normal(0, 5);
  theta ~ exponential(1);   // favors moderate overdispersion, >0

  // likelihood with log link
  y ~ neg_binomial_2_log(X * beta, theta);
}
"

```

```

fit <- stan(model_code = stan_code,
            data = stan_data,
            chains = 10,
            cores = 10,
            warmup = 2000,
            iter = 4000)

```

```
print(fit)
```

Inference for Stan model: anon_model.

10 chains, each with iter=4000; warmup=2000; thin=1;

post-warmup draws per chain=2000, total post-warmup draws=20000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
beta[1]	2.34	0.02	1.89	-1.25	1.11	2.31	3.52	6.19	6816	1
beta[2]	-0.05	0.00	0.03	-0.11	-0.07	-0.05	-0.03	0.01	7628	1
beta[3]	-0.24	0.00	0.33	-0.88	-0.44	-0.24	-0.04	0.40	8458	1
beta[4]	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	9309	1
beta[5]	0.05	0.00	0.05	-0.03	0.01	0.04	0.07	0.16	6790	1
theta	0.31	0.00	0.12	0.13	0.22	0.29	0.37	0.59	7745	1
lp__	-66.03	0.03	2.14	-71.28	-67.17	-65.65	-64.44	-63.05	4914	1

Samples were drawn using NUTS(diag_e) at Wed Oct 8 20:14:13 2025.

For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

Exercise 5 Experimenting with difficult posterior

The posterior below is somewhat difficult. The samples generate slowly and the effective sample size is relatively low.

Stan is excellent with default settings, but there are ways to improve efficiency even further for difficult problems.

Here's *one way* to think about efficiency: you want your effective *samples per second* (of your time) to be relatively large. Experiment with the several strategies to increase this measure of efficiency.

1. Rescale the predictors to have mean of zero and SD of one or 0.5. Or perhaps have a range from zero to one.
2. Increase the number of chains run in parallel.
3. Increase the number of iterations (and the warmup).
4. Increase the `adapt_delta` parameter. Search `?brm` for “`adapt_delta`” for more.

```
# load packages
library(brms)
library(posterior)

# load data
hks <- crdata::hks2013

# fit negative binomial model
f <- osvAll ~ troopLag + policeLag + militaryobserversLag +
  brv_AllLag + osvAllLagDum + incomp + epduration +
  lnthpop
fit_mcmc <- brm(
  f,
  data = hks,
  family = negbinomial,
  warmup = 5000,
  iter = 10000,
  chains = 4,
  cores = 4)

# print fit
fit_mcmc
```

Warning: There were 18 divergent transitions after warmup. Increasing adapt_delta above 0.8 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

Family: negbinomial
 Links: mu = log
 Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + incomp + epduration
 Data: hks (Number of observations: 3746)
 Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
 total post-warmup draws = 4000

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	-9.23	0.92	-11.02	-7.38	1.00	2262
troopLag	-0.52	0.07	-0.65	-0.38	1.00	2459
policeLag	-9.91	1.04	-11.92	-7.87	1.00	2415
militaryobserversLag	21.85	1.39	19.24	24.68	1.00	2537
brv_AllLag	0.00	0.00	-0.00	0.00	1.00	4081
osvAllLagDum	2.18	0.18	1.84	2.54	1.00	2727
incomp	2.37	0.19	2.00	2.75	1.00	2737
epduration	-0.00	0.00	-0.00	0.00	1.00	4735
lnpop	0.70	0.08	0.55	0.86	1.00	2365
	Tail_ESS					
Intercept	2236					
troopLag	2276					
policeLag	2288					
militaryobserversLag	2480					
brv_AllLag	2731					
osvAllLagDum	2645					
incomp	2706					
epduration	2696					
lnpop	2208					

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.06	0.00	0.05	0.06	1.00	3894	2272

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

How long it took.

We can grab the time (in seconds) it took to obtain the samples from the fit component of fit_mcmc.

```
# time (in seconds) it took to samples for each chain
times <- get_elapsed_time(fit_mcmc$fit)
times
```



```

      warmup sample
chain:1 11.624  5.113
chain:2 10.493  5.180
chain:3 11.204  4.232
chain:4 12.058  5.461

```

```

# find max of warmup + samples
t <- max(apply(times, 1, sum))
t

```

```
[1] 17.519
```

How many (effective) samples we generated.

We can compute the ESS using the `effective_samples()` function in `{bayestestR}` package. We need a way to think about the ESS overall, so I just averaged the bulk ESS across the parameters.

```

# load package; some helpful things here!
library(bayestestR)

# compute ess for each parameter
ess <- effective_sample(fit_mcmc)
ess

```

	Parameter	ESS	ESS_tail
1	b_Intercept	2262	2236
2	b_troopLag	2459	2276
3	b_policeLag	2415	2288
4	b_militaryobserversLag	2537	2480
5	b_brv_AllLag	4081	2731
6	b_osvAllLagDum	2727	2645
7	b_incomp	2737	2706
8	b_epduration	4735	2696
9	b_lntpop	2365	2208

```

# compute the average ess for a single number
avg_ess <- mean(ess$ESS)
avg_ess

```

```
[1] 2924.222
```

Effective samples per second.

And then we can compute the ratio—effective samples per second.

```
avg_ess/t
```

```
[1] 166.9172
```

To make this easy, we can make a function to compute the effective samples per second.

```
sps <- function(fit) {  
  times <- get_elapsed_time(fit$fit)  
  t <- max(apply(times, 1, sum))  
  ess <- effective_sample(fit)  
  avg_ess <- mean(ess$ESS)  
  list("sps" = avg_ess/t,  
       "ess" = ess,  
       "times" = times)  
}
```

Solution.

Strategy 1: Rescale predictors.

```
rs <- \(x) arm::rescale(x) # alias  
  
f_rs <- osvAll ~ rs(troopLag) + rs(policeLag) + rs(militaryobserversLag) +  
  rs(brv_AllLag) + rs(osvAllLagDum) + rs(incomp) + rs(epduration) +  
  rs(lntpop)  
  
fit_mcmc_1 <- brm(  
  f_rs,  
  data = hks,  
  family = negbinomial,  
  warmup = 5000,  
  iter = 10000,  
  chains = 4,  
  cores = 4)
```

```
print(fit_mcmc_1)
```

```
Family: negbinomial
Links: mu = log
Formula: osvAll ~ rs(troopLag) + rs(policeLag) + rs(militaryobserversLag) + rs(brv_AllLag) +
  Data: hks (Number of observations: 3746)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000
```

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	1.85	0.07	1.71	1.99	1.00	4998
rstroopLag	-3.00	0.38	-3.71	-2.22	1.00	3790
rspoliceLag	-2.81	0.30	-3.38	-2.23	1.00	5043
rsmilitaryobserversLag	4.63	0.29	4.09	5.25	1.00	4025
rsbrv_AllLag	0.24	0.20	-0.08	0.69	1.00	4493
rsosvAllLagDum	2.19	0.18	1.85	2.55	1.00	4307
rsincomp	2.37	0.18	2.01	2.73	1.00	4144
rsepduration	-0.06	0.15	-0.36	0.25	1.00	4582
rslntpop	1.71	0.19	1.33	2.08	1.00	3527

Tail_ESS

Intercept	2955
rstroopLag	3111
rspoliceLag	2863
rsmilitaryobserversLag	2917
rsbrv_AllLag	2466
rsosvAllLagDum	2991
rsincomp	3332
rsepduration	3294
rslntpop	3302

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.06	0.00	0.05	0.06	1.00	5526	2897

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
sps(fit_mcmc_1)
```

```
$sps
```

```
[1] 655.5303
```

```
$ess
```

	Parameter	ESS	ESS_tail
1	b_Intercept	4998	2955
2	b_rstroopLag	3790	3111
3	b_rspoliceLag	5043	2863
4	b_rsmilitaryobserversLag	4025	2917
5	b_rsbrv_AllLag	4493	2466
6	b_rsosvAllLagDum	4307	2991
7	b_rsincomp	4144	3332
8	b_rsepduration	4582	3294
9	b_rslntpop	3527	3302

```
$times
```

	warmup	sample
chain:1	3.286	2.993
chain:2	3.152	3.156
chain:3	3.374	3.221
chain:4	3.272	2.953

Strategy 2: Increase number of chains.

I have 12 cores on my computer, so I can easily run 10 chains in parallel rather than just 4.

```
fit_mcmc_2 <- brm(  
  f,  
  data = hks,  
  family = negbinomial,  
  warmup = 5000,  
  iter = 10000,  
  chains = 10,  
  cores = 10)
```

```
print(fit_mcmc_2)
```

Warning: Parts of the model have not converged (some Rhats are > 1.05). Be careful when analysing the results! We recommend running more iterations and/or setting stronger priors.

Warning: There were 3733 divergent transitions after warmup. Increasing adapt_delta above 0.8 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

```

Family: negbinomial
Links: mu = log
Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + :
Data: hks (Number of observations: 3746)
Draws: 10 chains, each with iter = 2000; warmup = 1000; thin = 1;
total post-warmup draws = 10000

```

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	22.12	40.55	-10.79	99.33	2.00	14
troopLag	-0.32	0.58	-1.03	1.23	2.50	12
policeLag	-5.86	5.09	-11.66	1.99	1.98	14
militaryobserversLag	13.11	10.82	-1.86	24.47	1.96	14
brv_AllLag	0.08	0.10	-0.00	0.20	1.99	14
osvAllLagDum	0.88	1.65	-1.65	2.50	1.97	14
incomp	1.31	1.56	-1.57	2.69	1.99	14
epduration	-0.46	0.59	-1.65	0.00	1.99	14
lnpop	0.42	0.71	-1.04	1.42	2.49	12

	Tail_ESS
Intercept	10
troopLag	11
policeLag	17
militaryobserversLag	17
brv_AllLag	10
osvAllLagDum	15
incomp	12
epduration	10
lnpop	13

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.75	0.95	0.06	2.77	1.99	14	10

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
sps(fit_mcmc_2)
```

```

$sps
[1] 0.5259391

```

```
$ess
      Parameter ESS ESS_tail
1      b_Intercept 14      10
2      b_troopLag 12      11
3      b_policeLag 14      17
4 b_militaryobserversLag 14      17
5      b_brv_AllLag 14      10
6      b_osvAllLagDum 14      15
7      b_incomp 14      12
8      b_epduration 14      10
9      b_lntpop 12      13
```

```
$times
      warmup sample
chain:1  2.037  5.287
chain:2 20.806  4.968
chain:3 14.024  5.583
chain:4  2.925  4.752
chain:5  2.409  4.826
chain:6 12.072  5.064
chain:7 11.038  4.430
chain:8  2.189 22.226
chain:9  9.411  5.623
chain:10 14.465  5.260
```

Strategy 3: Increase number iterations.

```
fit_mcmc_3 <- brm(
  f,
  data = hks,
  family = negbinomial,
  warmup = 25000, # up from 5000
  iter = 50000, # up from 10000
  chains = 4,
  cores = 4)
```

```
print(fit_mcmc_3)
```

Warning: Parts of the model have not converged (some Rhats are > 1.05). Be careful when analysing the results! We recommend running more iterations and/or setting stronger priors.

Warning: There were 903 divergent transitions after warmup. Increasing adapt_delta above 0.8 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

Family: negbinomial
 Links: mu = log
 Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + :
 Data: hks (Number of observations: 3746)
 Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
 total post-warmup draws = 20000

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	0.96	23.64	-10.87	79.20	1.59	7
troopLag	-0.42	0.40	-0.88	0.60	1.59	7
policeLag	-7.41	4.43	-11.78	0.13	1.54	7
militaryobserversLag	16.13	10.03	-1.32	24.61	1.59	7
brv_AllLag	0.02	0.06	-0.00	0.20	1.47	8
osvAllLagDum	1.80	0.70	0.34	2.51	1.59	7
incomp	1.53	1.48	-1.35	2.71	1.59	7
epduration	-0.09	0.29	-1.12	0.00	1.59	7
lnpop	0.40	0.55	-0.73	0.85	1.59	7
	Tail_ESS					
Intercept	11					
troopLag	11					
policeLag	12					
militaryobserversLag	11					
brv_AllLag	20					
osvAllLagDum	11					
incomp	19					
epduration	11					
lnpop	13					

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.23	0.45	0.04	1.53	1.55	9	11

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
sps(fit_mcmc_3)
```

```
$sps
```

```
[1] 0.01371012
```

```
$ess
```

	Parameter	ESS	ESS_tail
1	b_Intercept	7	11
2	b_troopLag	7	11
3	b_policeLag	7	12
4	b_militaryobserversLag	7	11
5	b_brv_AllLag	8	20
6	b_osvAllLagDum	7	11
7	b_incomp	7	19
8	b_epduration	7	11
9	b_lntpop	7	13

```
$times
```

	warmup	sample
chain:1	27.483	15.861
chain:2	26.324	15.961
chain:3	24.197	15.153
chain:4	9.961	508.715

Strategy 4a: Increase adapt_delta.

```
fit_mcmc_4a <- brm(  
  f,  
  data = hks,  
  family = negbinomial,  
  warmup = 5000,  
  iter = 10000,  
  control = list(adapt_delta = 0.95), # from default of 0.8  
  chains = 4,  
  cores = 4)
```

```
print(fit_mcmc_4a)
```

Warning: Parts of the model have not converged (some Rhats are > 1.05). Be careful when analysing the results! We recommend running more iterations and/or setting stronger priors.

Warning: There were 958 divergent transitions after warmup. Increasing adapt_delta above 0.95 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

Family: negbinomial
 Links: mu = log
 Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + :
 Data: hks (Number of observations: 3746)
 Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
 total post-warmup draws = 4000

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	20.46	51.40	-10.78	109.52	1.59	7
troopLag	-0.46	0.12	-0.65	-0.28	1.58	7
policeLag	-6.99	5.11	-11.73	1.73	1.57	7
militaryobserversLag	16.55	9.29	0.59	24.53	1.59	7
brv_AllLag	0.05	0.08	-0.00	0.19	1.59	7
osvAllLagDum	1.28	1.56	-1.41	2.52	1.58	7
incomp	1.94	0.76	0.66	2.71	1.59	7
epduration	-0.38	0.66	-1.52	0.00	1.59	7
lnpop	0.04	1.15	-1.94	0.84	1.54	7
	Tail_ESS					
Intercept	11					
troopLag	12					
policeLag	14					
militaryobserversLag	11					
brv_AllLag	11					
osvAllLagDum	11					
incomp	12					
epduration	11					
lnpop	17					

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.77	1.23	0.06	2.90	1.59	7	11

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
sps(fit_mcmc_4a)
```

```
$sps
```

```
[1] 0.2579219
```

```
$ess
```

	Parameter	ESS	ESS_tail
1	b_Intercept	7	11
2	b_troopLag	7	12
3	b_policeLag	7	14
4	b_militaryobserversLag	7	11
5	b_brv_AllLag	7	11
6	b_osvAllLagDum	7	11
7	b_incomp	7	12
8	b_epduration	7	11
9	b_lntpop	7	17

```
$times
```

	warmup	sample
chain:1	20.818	5.590
chain:2	8.072	3.462
chain:3	12.798	8.785
chain:4	17.995	9.145

Strategy 4b: Increase adapt_delta even more.

```
fit_mcmc_4b <- brm(  
  f,  
  data = hks,  
  family = negbinomial,  
  warmup = 5000,  
  iter = 10000,  
  control = list(adapt_delta = 0.99), # from default of 0.8  
  chains = 4,  
  cores = 4)
```

```
print(fit_mcmc_4b)
```

```
Family: negbinomial  
Links: mu = log
```

Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + :
 Data: hks (Number of observations: 3746)
 Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
 total post-warmup draws = 4000

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS
Intercept	-9.25	0.90	-11.00	-7.50	1.00	2739
troopLag	-0.53	0.07	-0.65	-0.39	1.00	2320
policeLag	-9.93	1.01	-11.91	-8.01	1.00	3266
militaryobserversLag	21.88	1.35	19.38	24.61	1.00	2555
brv_AllLag	0.00	0.00	-0.00	0.00	1.00	3513
osvAllLagDum	2.18	0.18	1.83	2.54	1.00	3282
incomp	2.37	0.18	2.01	2.73	1.00	3139
epduration	-0.00	0.00	-0.00	0.00	1.00	4889
lnpop	0.71	0.08	0.55	0.86	1.00	2871
	Tail_ESS					
Intercept	2339					
troopLag	2115					
policeLag	2734					
militaryobserversLag	2521					
brv_AllLag	2218					
osvAllLagDum	2676					
incomp	2711					
epduration	2972					
lnpop	2600					

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
shape	0.06	0.00	0.05	0.06	1.00	3472	2635

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

```
sps(fit_mcmc_4b)
```

```
$sps
```

```
[1] 69.67212
```

```
$ess
```

```
Parameter ESS ESS_tail
```

1	b_Intercept	2739	2339
2	b_troopLag	2320	2115
3	b_policeLag	3266	2734
4	b_militaryobserversLag	2555	2521
5	b_brv_AllLag	3513	2218
6	b_osvAllLagDum	3282	2676
7	b_incomp	3139	2711
8	b_epduration	4889	2972
9	b_lntpop	2871	2600

\$times

	warmup	sample
chain:1	27.277	10.635
chain:2	35.219	10.350
chain:3	31.500	9.762
chain:4	26.140	9.791